

# Trabajo Practico 3

---

## Optimizacion de consultas sobre Food Store

**Base de Datos II - Unidad 2 - Semana 3**

Estudiante: Santiago Barretto

Repositorio: DB\_Santiago\_Barretto

[https://github.com/Santiago-Btto/DB\\_Santiago\\_Barretto](https://github.com/Santiago-Btto/DB_Santiago_Barretto)

Commit del dia 7/9/26

Comandos utilizados para el uso de psql:

```
PS C:\Users\esteb\Desktop\faku\3r Semestre\BDII\UI\DB_Santiago_Barretto> $env:Path = 'C:\Program Files\PostgreSQL\18\bin;' + $env:Path
PS C:\Users\esteb\Desktop\faku\3r Semestre\BDII\UI\DB_Santiago_Barretto> psql --version
psql (PostgreSQL) 18.4
PS C:\Users\esteb\Desktop\faku\3r Semestre\BDII\UI\DB_Santiago_Barretto> pg_dump --version
pg_dump (PostgreSQL) 18.4
PS C:\Users\esteb\Desktop\faku\3r Semestre\BDII\UI\DB_Santiago_Barretto> createdb --version
createdb (PostgreSQL) 18.4
```

## 1 Objetivo y protocolo de trabajo

El TP3 optimiza consultas reales sobre Food Store con volumen suficiente para medir cambios en el plan. La IA propone scripts, reescrituras e indices; cada decision se acepta solo despues de leerla y contrastarla contra EXPLAIN ANALYZE.

### Protocolo de seguridad

| Paso        | Aplicacion                                                                              |
|-------------|-----------------------------------------------------------------------------------------|
| Copia       | Crear y usar food_store_tp3_lab; nunca ejecutar sobre la base original.                 |
| Respaldo    | Ejecutar pg_dump sobre la copia antes de cambios estructurales o carga masiva.          |
| Transaccion | La carga y los indices tienen controles de error; revisar la salida antes de continuar. |
| Evidencia   | Conservar las salidas de tp3/evidencia/ y transcribir solo resultados demostrados.      |

### Ejecucion reproducible

Desde la carpeta tp3 se ejecuta el siguiente comando. Genera preflight, carga, planes antes y despues, indices, equivalencias y validacion.

```
.\run_mediciones.ps1 -Database food_store_tp3_lab -Lote SB2026TP3 -PrepararDatos
```

## 2 Parte 1 Carga masiva

01\_carga\_masiva.sql usa generate\_series dentro de una transaccion y no elimina ni modifica filas existentes. Al finalizar, ANALYZE actualiza estadisticas de cliente, producto, pedido y detalle\_pedido.

| Entidad        | Cantidad | Controles                                                             |
|----------------|----------|-----------------------------------------------------------------------|
| Producto       | 50.000   | Categorias existentes, precio 500 a 5000, stock 0 a 200.              |
| Cliente        | 20.000   | Email unico e identificacion por lote.                                |
| Pedido         | 200.000  | Cliente existente, fecha generada y forma de pago valida.             |
| Detalle pedido | 400.000  | Dos productos distintos por pedido, PK compuesta y valores positivos. |

La validacion posterior exige estos conteos, los rangos de producto, la unicidad de email y la ausencia de referencias huerfanas.

### 3 Parte 2 Consultas lentas y optimizacion medida

Las mismas tres consultas se ejecutan antes y despues. Los indices son hipotesis de mejora: la decision final depende del nodo elegido y del tiempo real del plan.

| Consulta | Patron                                                           | Cambio propuesto                                                     |
|----------|------------------------------------------------------------------|----------------------------------------------------------------------|
| Q1       | Productos activos por categoria, rango de precio, orden y LIMIT. | Indice parcial: categoria, precio DESC e identificador para activos. |
| Q2       | Pedidos recientes por forma de pago, fecha DESC y LIMIT.         | Indice por forma de pago y fecha DESC.                               |
| Q3       | Productos mas vendidos de los ultimos siete dias.                | Indice por fecha de pedido para acotar el rango antes de los joins.  |

#### Tabla de resultados

| Consulta | Plan antes                             | Cambio                     | Plan despues                 | Mejora            |
|----------|----------------------------------------|----------------------------|------------------------------|-------------------|
| Q1       | Bitmap Heap Scan y Sort; 44.727 ms     | Indice parcial de producto | Bitmap Index Scan; 45.011 ms | 0.994x: no mejoro |
| Q2       | Parallel Seq Scan y Sort; 26.940 ms    | Indice forma_pago y fecha  | Index Scan; 0.553 ms         | 48.72x            |
| Q3       | Parallel Seq Scan de pedido; 68.061 ms | Indice fecha de pedido     | Bitmap Index Scan; 52.491 ms | 1.30x             |

Mejora x = tiempo total antes / tiempo total despues. cost es una estimacion del planificador; no representa milisegundos. Q1 no se acepta como mejora: aunque el indice se uso, la consulta que calcula la categoria siguio haciendo un Seq Scan y el tiempo total fue ligeramente mayor.

### 4 Parte 3 Lectura critica de un plan

Elegir un plan posterior real, pegarlo junto con la respuesta literal de IA y contrastar cada afirmacion. Distinguir cost, actual time, rows, loops, buffers y Rows Removed by Filter.

| Afirmacion de IA                                                                        | Correcta | Evidencia del plan real                                                                               |
|-----------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------|
| El indice por forma de pago y fecha evita leer todos los pedidos y evita un Sort final. | Si       | Index Scan idx_tp3_pedido_forma_pago_fecha; no hay nodo Sort; Index Cond contiene forma_pago y fecha. |
| El costo final 143.13 representa que la consulta tardo 143.13 milisegundos.             | No       | cost=0.42..143.13 es estimado. El tiempo real es Execution Time: 0.553 ms.                            |
| Los 200 resultados se obtuvieron con pocas lecturas de buffers.                         | Si       | Buffers: shared hit=200 read=4 y 200 filas devueltas por Index Scan.                                  |

## 5 Parte 4 Consultas con especificacion precisa

### Spec 1 Resumen por categoria

Devuelve, para cada categoria activa, identificador, nombre, cantidad de productos activos y valor de lista total. Incluye categorias sin productos con cero; ordena por cantidad descendente, nombre e identificador. La version IA usa LEFT JOIN y agregacion; la alternativa propia usa subconsultas correlacionadas.

### Spec 2 Productos sin ventas recientes

Devuelve productos y categorias activos que no tuvieron detalles asociados a pedidos de los ultimos 90 dias. La version IA usa NOT IN y la alternativa propia NOT EXISTS. detalle\_pedido.id\_producto no admite NULL por su clave primaria compuesta.

### Equivalencia formal

| Control                | Esperado | Resultado real    |
|------------------------|----------|-------------------|
| Spec 1 IA menos propia | 0 filas  | 0 filas distintas |
| Spec 1 propia menos IA | 0 filas  | 0 filas distintas |
| Spec 2 IA menos propia | 0 filas  | 0 filas distintas |
| Spec 2 propia menos IA | 0 filas  | 0 filas distintas |

## 6 Parte 5 Competencia de optimizacion

La consulta comun lista productos y categorias activos con precio entre 1500 y 4500, ordenados por precio descendente e identificador, con LIMIT 100. Se mide antes de la estrategia y luego se repite en la misma copia.

| Equipo            | Estrategia                                             | Antes ms | Despues ms | Mejora x |
|-------------------|--------------------------------------------------------|----------|------------|----------|
| Santiago Barretto | Indice parcial precio DESC e identificador con INCLUDE | 22.246   | 0.549      | 40.52x   |

## 7 Declaracion de Uso de IA

La IA se utilizo para proponer scripts y documentacion. Se revisa cada propuesta como hipotesis y la aceptacion final depende de PostgreSQL y de la evidencia conservada.

| Uso          | Especificacion resumida                                                                  | Decision y verificacion                                                   |
|--------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Carga masiva | Generar 50k productos, 20k clientes, 200k pedidos y detalles con PK, FK, UNIQUE y CHECK. | Aceptada y ejecutada: conteos reales validados sin referencias huérfanas. |
| Q1 Q2 Q3     | Proponer consultas medibles y un indice por patron, sin afirmar mejora.                  | Q2 y Q3 confirmados; Q1 documentado como no mejorado.                     |
| Parte 4      | Una consulta resumen y una subconsulta, con filtros activos y EXCEPT bidireccional.      | Aceptadas: los cuatro controles devolvieron cero.                         |
| Parte 3 y 5  | Explicar plan real y proponer estrategia para consulta comun.                            | Parte 3 contrastada contra Q2; competencia confirmada con mejora 40.52x.  |

## 8 Checklist antes de entregar

- Ejecute el runner sobre una copia de laboratorio.
- Guarde planes antes, indices y planes despues en evidencia/.
- Complete la tabla de resultados con nodos y tiempos reales, no estimados.
- Pegue un plan posterior y la explicacion literal de IA en la Parte 3.
- Confirme los cuatro EXCEPT de Parte 4 con resultado cero.
- Registre propuestas aceptadas y descartadas en la DUIA.
- Puedo explicar cada script e indice incluido en la entrega.

# Anexo A - Planes EXPLAIN ANALYZE

Q1: medición antes del índice o cambio propuesto

## Q1 - plan completo antes

### QUERY PLAN

```
-----
Limit  (cost=5688.76..5689.01 rows=100 width=47) (actual time=44.588..44.597 rows=100.00 loops=1)
  Buffers: shared hit=2735 read=10
  -> Sort  (cost=5688.76..5704.58 rows=6330 width=47) (actual time=44.588..44.592 rows=100.00 loops=1)
    Sort Key: p.precio_lista DESC, p.id_producto
    Sort Method: top-N heapsort  Memory: 37kB
    Buffers: shared hit=2735 read=10
    -> Nested Loop  (cost=3398.18..5446.83 rows=6330 width=47) (actual time=40.349..43.595 rows=6666.00 loops=1)
      Buffers: shared hit=2729 read=10
      -> Limit  (cost=3205.25..3205.26 rows=1 width=16) (actual time=39.906..39.908 rows=1.00 loops=1)
        Buffers: shared hit=1819
        -> Sort  (cost=3205.25..3205.27 rows=5 width=16) (actual time=39.906..39.907 rows=1.00 loops=1)
          Sort Key: (count(*)) DESC, p_1.id_categoria
          Sort Method: top-N heapsort  Memory: 25kB
          Buffers: shared hit=1819
          -> GroupAggregate  (cost=3203.39..3205.23 rows=5 width=16) (actual time=37.143..39.901 rows=5.00 loops=1)
            Group Key: p_1.id_categoria
            Buffers: shared hit=1819
            -> Sort  (cost=3203.39..3203.99 rows=238 width=8) (actual time=36.401..37.702 rows=47500.00 loops=1)
              Sort Key: p_1.id_categoria
              Sort Method: quicksort  Memory: 1537kB
              Buffers: shared hit=1819
              -> Seq Scan on producto p_1  (cost=0.00..3194.00 rows=238 width=8) (actual time=1.189..33.740 rows=47500.00 loops=1)
                Filter: (activo AND ("left"((nombre)::text, length(format('TP3_%s_PRODUCTO_':::text, current_setting('app.tp3_lote':
:text)))) = format('TP3_%s_PRODUCTO_':::text, current_setting('app.tp3_lote':::text))))
                Rows Removed by Filter: 2500
                Buffers: shared hit=1819
            -> Bitmap Heap Scan on producto p  (cost=192.92..2178.26 rows=6330 width=55) (actual time=0.440..3.143 rows=6666.00 loops=1)
              Recheck Cond: ((id_categoria = p_1.id_categoria) AND activo)
              Filter: ((precio_lista >= '1500':::numeric) AND (precio_lista <= '4500':::numeric))
              Rows Removed by Filter: 3334
              Heap Blocks: exact=910
              Buffers: shared hit=910 read=10
              -> Bitmap Index Scan on idx_producto_categoria_activo  (cost=0.00..191.34 rows=9505 width=0) (actual time=0.351..0.351 rows=10000.00 loops
=1)
                Index Cond: ((id_categoria = p_1.id_categoria) AND (activo = true))
                Index Searches: 1
                Buffers: shared read=10

Planning:
  Buffers: shared hit=160 read=2 dirtied=2 written=2
Planning Time: 2.725 ms
Execution Time: 44.727 ms
(39 rows)
```

## Anexo A - Planes EXPLAIN ANALYZE

Q2: medición antes del índice o cambio propuesto

### Q2 - plan completo antes

#### QUERY PLAN

```
-----
Limit  (cost=6703.55..6726.34 rows=200 width=28) (actual time=23.833..26.912 rows=200.00 loops=1)
  Buffers: shared hit=2980
  -> Gather Merge  (cost=6703.55..8535.28 rows=16072 width=28) (actual time=23.832..26.901 rows=200.00 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    Buffers: shared hit=2980
    -> Sort  (cost=5703.54..5727.17 rows=9454 width=28) (actual time=10.937..10.947 rows=100.00 loops=2)
      Sort Key: fecha DESC
      Sort Method: top-N heapsort  Memory: 48kB
      Buffers: shared hit=2980
      Worker 0:  Sort Method: quicksort  Memory: 25kB
      -> Parallel Seq Scan on pedido ped  (cost=0.00..5294.94 rows=9454 width=28) (actual time=0.873..10.411 rows=8219.50 loops=2)
        Filter: (((forma_pago = 'TARJETA'::forma_pago_enum) AND (fecha >= (CURRENT_TIMESTAMP - '90 days'::interval)))
        Rows Removed by Filter: 91780
        Buffers: shared hit=2942

Planning:
  Buffers: shared hit=70 read=2 dirtied=1
Planning Time: 1.376 ms
Execution Time: 26.940 ms
(19 rows)
```

## Anexo A - Planes EXPLAIN ANALYZE

Q3: medición antes del índice o cambio propuesto

### Q3 - plan completo antes

#### QUERY PLAN

```
-----
Limit  (cost=14375.19..14375.24 rows=20 width=46) (actual time=64.942..67.882 rows=20.00 loops=1)
  Buffers: shared hit=28753 read=136 written=77
  -> Sort  (cost=14375.19..14392.79 rows=7042 width=46) (actual time=64.941..67.879 rows=20.00 loops=1)
    Sort Key: (sum(det.cantidad)) DESC, p.id_producto
    Sort Method: top-N heapsort  Memory: 27kB
    Buffers: shared hit=28753 read=136 written=77
    -> HashAggregate  (cost=14117.38..14187.80 rows=7042 width=46) (actual time=63.361..67.289 rows=6977.00 loops=1)
      Group Key: p.id_producto
      Batches: 1  Memory Usage: 793kB
      Buffers: shared hit=28753 read=136 written=77
      -> Gather  (cost=6028.35..14082.17 rows=7042 width=42) (actual time=26.189..64.173 rows=7258.00 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        Buffers: shared hit=28753 read=136 written=77
        -> Nested Loop  (cost=5028.35..12377.97 rows=4142 width=42) (actual time=13.571..48.944 rows=3629.00 loops=2)
          Buffers: shared hit=28753 read=136 written=77
          -> Parallel Hash Join  (cost=5028.06..10940.66 rows=4358 width=12) (actual time=13.529..38.617 rows=3834.00 loops=2)
            Hash Cond: (det.id_pedido = ped.id_pedido)
            Buffers: shared hit=5884
            -> Parallel Seq Scan on detalle_pedido det  (cost=0.00..5294.94 rows=235294 width=20) (actual time=0.008..10.413 rows=200000.0
0 loops=2)
              Buffers: shared hit=2942
            -> Parallel Hash  (cost=5000.82..5000.82 rows=2179 width=8) (actual time=13.313..13.314 rows=1917.00 loops=2)
              Buckets: 4096  Batches: 1  Memory Usage: 224kB
              Buffers: shared hit=2942
            -> Parallel Seq Scan on pedido ped  (cost=0.00..5000.82 rows=2179 width=8) (actual time=0.225..13.047 rows=1917.00 loops
=2)
              Filter: (fecha >= (CURRENT_TIMESTAMP - '7 days'::interval))
              Rows Removed by Filter: 98083
              Buffers: shared hit=2942
          -> Index Scan using producto_pkey on producto p  (cost=0.29..0.33 rows=1 width=38) (actual time=0.002..0.002 rows=0.95 loops=7668)
            Index Cond: (id_producto = det.id_producto)
            Filter: activo
            Rows Removed by Filter: 0
            Index Searches: 7668
            Buffers: shared hit=22869 read=136 written=77

Planning:
  Buffers: shared hit=53 read=16
Planning Time: 9.723 ms
Execution Time: 68.061 ms
(38 rows)
```

# Anexo A - Planes EXPLAIN ANALYZE

Q1: medición posterior al cambio aplicado

## Q1 - plan completo después

### QUERY PLAN

```
-----
Limit  (cost=5649.73..5649.98 rows=100 width=47) (actual time=44.873..44.882 rows=100.00 loops=1)
  Buffers: shared hit=2734 read=36
  -> Sort  (cost=5649.73..5665.57 rows=6335 width=47) (actual time=44.872..44.876 rows=100.00 loops=1)
    Sort Key: p.precio_lista DESC, p.id_producto
    Sort Method: top-N heapsort  Memory: 37kB
    Buffers: shared hit=2734 read=36
    -> Nested Loop  (cost=3414.39..5407.61 rows=6335 width=47) (actual time=41.802..43.859 rows=6666.00 loops=1)
      Buffers: shared hit=2728 read=36
      -> Limit  (cost=3205.20..3205.20 rows=1 width=16) (actual time=40.801..40.803 rows=1.00 loops=1)
        Buffers: shared hit=1819
        -> Sort  (cost=3205.20..3205.21 rows=5 width=16) (actual time=40.800..40.801 rows=1.00 loops=1)
          Sort Key: (count(*)) DESC, p_1.id_categoria
          Sort Method: top-N heapsort  Memory: 25kB
          Buffers: shared hit=1819
          -> GroupAggregate  (cost=3203.35..3205.18 rows=5 width=16) (actual time=37.983..40.791 rows=5.00 loops=1)
            Group Key: p_1.id_categoria
            Buffers: shared hit=1819
            -> Sort  (cost=3203.35..3203.94 rows=237 width=8) (actual time=37.249..38.555 rows=47500.00 loops=1)
              Sort Key: p_1.id_categoria
              Sort Method: quicksort  Memory: 1537kB
              Buffers: shared hit=1819
              -> Seq Scan on producto p_1  (cost=0.00..3194.00 rows=237 width=8) (actual time=1.135..34.680 rows=47500.00 loops=1)
                Filter: (activo AND ("left"((nombre)::text, length(format('TP3_%s_PRODUCTO_':::text, current_setting('app.tp3_lote':
:text)))) = format('TP3_%s_PRODUCTO_':::text, current_setting('app.tp3_lote':::text))))
                Rows Removed by Filter: 2500
                Buffers: shared hit=1819
            -> Bitmap Heap Scan on producto p  (cost=209.19..2139.05 rows=6335 width=55) (actual time=0.996..2.031 rows=6666.00 loops=1)
              Recheck Cond: ((p_1.id_categoria = id_categoria) AND (precio_lista >= '1500':::numeric) AND (precio_lista <= '4500':::numeric) AND activo)
              Heap Blocks: exact=909
              Buffers: shared hit=909 read=36
              -> Bitmap Index Scan on idx_tp3_producto_categoria_precio_activo  (cost=0.00..207.60 rows=6335 width=0) (actual time=0.905..0.905 rows=666
6.00 loops=1)
                Index Cond: ((id_categoria = p_1.id_categoria) AND (precio_lista >= '1500':::numeric) AND (precio_lista <= '4500':::numeric))
                Index Searches: 1
                Buffers: shared read=36

Planning:
  Buffers: shared hit=194 read=1
Planning Time: 2.380 ms
Execution Time: 45.011 ms
(37 rows)
```

# Anexo A - Planes EXPLAIN ANALYZE

Q2: medición posterior al cambio aplicado

## Q2 - plan completo después

```

-----
QUERY PLAN
-----
Limit  (cost=0.42..143.13 rows=200 width=28) (actual time=0.061..0.540 rows=200.00 loops=1)
  Buffers: shared hit=200 read=4
  -> Index Scan using idx_tp3_pedido_forma_pago_fecha on pedido ped  (cost=0.42..11645.72 rows=16321 width=28) (actual time=0.060..0.529 rows=200.00 loops=1)
    Index Cond: ((forma_pago = 'TARJETA'::forma_pago_enum) AND (fecha >= (CURRENT_TIMESTAMP - '90 days'::interval)))
    Index Searches: 1
    Buffers: shared hit=200 read=4
Planning:
  Buffers: shared hit=117 read=2
Planning Time: 2.437 ms
Execution Time: 0.553 ms
(10 rows)
```

## Anexo A - Planes EXPLAIN ANALYZE

Q3: medición posterior al cambio aplicado

### Q3 - plan completo después

#### QUERY PLAN

```
-----
Limit  (cost=12570.47..12570.52 rows=20 width=46) (actual time=49.650..52.330 rows=20.00 loops=1)
  Buffers: shared hit=27174 read=11
  -> Sort  (cost=12570.47..12587.96 rows=6999 width=46) (actual time=49.649..52.328 rows=20.00 loops=1)
    Sort Key: (sum(det.cantidad)) DESC, p.id_producto
    Sort Method: top-N heapsort  Memory: 27kB
    Buffers: shared hit=27174 read=11
    -> HashAggregate  (cost=12314.24..12384.23 rows=6999 width=46) (actual time=48.075..51.761 rows=6977.00 loops=1)
      Group Key: p.id_producto
      Batches: 1  Memory Usage: 793kB
      Buffers: shared hit=27174 read=11
      -> Gather  (cost=4236.32..12279.24 rows=6999 width=42) (actual time=4.630..48.753 rows=7258.00 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        Buffers: shared hit=27174 read=11
        -> Nested Loop  (cost=3236.32..10579.34 rows=4117 width=42) (actual time=1.981..33.195 rows=3629.00 loops=2)
          Buffers: shared hit=27174 read=11
          -> Parallel Hash Join  (cost=3236.03..9148.63 rows=4338 width=12) (actual time=1.962..25.745 rows=3834.00 loops=2)
            Hash Cond: (det.id_pedido = ped.id_pedido)
            Buffers: shared hit=4169 read=11
            -> Parallel Seq Scan on detalle_pedido det  (cost=0.00..5294.94 rows=235294 width=20) (actual time=0.006..10.149 rows=200000.0
0 loops=2)
              Buffers: shared hit=2942
            -> Parallel Hash  (cost=3208.91..3208.91 rows=2169 width=8) (actual time=1.875..1.876 rows=1917.00 loops=2)
              Buckets: 4096  Batches: 1  Memory Usage: 192kB
              Buffers: shared hit=1227 read=11
              -> Parallel Bitmap Heap Scan on pedido ped  (cost=73.00..3208.91 rows=2169 width=8) (actual time=0.517..3.395 rows=3834.
00 loops=1)
                Recheck Cond: (fecha >= (CURRENT_TIMESTAMP - '7 days'::interval))
                Heap Blocks: exact=1224
                Buffers: shared hit=1227 read=11
                -> Bitmap Index Scan on idx_tp3_pedido_fecha  (cost=0.00..72.08 rows=3687 width=0) (actual time=0.390..0.390 rows=
3834.00 loops=1)
                  Index Cond: (fecha >= (CURRENT_TIMESTAMP - '7 days'::interval))
                  Index Searches: 1
                  Buffers: shared hit=3 read=11
            -> Index Scan using producto_pkey on producto p  (cost=0.29..0.33 rows=1 width=38) (actual time=0.002..0.002 rows=0.95 loops=7668)
              Index Cond: (id_producto = det.id_producto)
              Filter: activo
              Rows Removed by Filter: 0
              Index Searches: 7668
              Buffers: shared hit=23005

Planning:
  Buffers: shared hit=74 read=3
Planning Time: 1.753 ms
Execution Time: 52.491 ms
(42 rows)
```

# Anexo B - Evidencia de Parte 3

Plan posterior, prompt y respuesta de IA usados para la lectura crítica

## Plan posterior elegido: Q2

```

                                QUERY PLAN
-----
Limit  (cost=0.42..143.13 rows=200 width=28) (actual time=0.061..0.540 rows=200.00 loops=1)
  Buffers: shared hit=200 read=4
  ->  Index Scan using idx_tp3_pedido_forma_pago_fecha on pedido ped  (cost=0.42..11645.72 rows=16321 width=28) (actual time=0.060..0.529 rows=200.00 loops=1)
        Index Cond: ((forma_pago = 'TARJETA'::forma_pago_enum) AND (fecha >= (CURRENT_TIMESTAMP - '90 days'::interval)))
        Index Searches: 1
        Buffers: shared hit=200 read=4
Planning:
  Buffers: shared hit=117 read=2
Planning Time: 2.437 ms
Execution Time: 0.553 ms
(10 rows)
```

## Prompt enviado a la IA

Explicá este plan EXPLAIN ANALYZE de PostgreSQL nodo por nodo, sin contexto adicional. Indicá qué filtros usa, si necesita ordenar, qué representan cost, actual time, rows, loops y buffers, y no confundas costos estimados con milisegundos reales.

## Respuesta de IA contrastada

El plan utiliza el índice compuesto idx\_tp3\_pedido\_forma\_pago\_fecha para localizar pedidos de TARJETA dentro del rango de 90 días. El Index Cond contiene ambos filtros. Como el índice conserva el orden de fecha descendente requerido por ORDER BY, PostgreSQL puede devolver los primeros 200 registros sin un nodo Sort. El costo 0.42..143.13 es una estimación del planificador; la medida real debe leerse en Execution Time, que es 0.553 ms. El acceso registró 200 buffers en memoria y 4 lecturas, por lo que no se recorrió toda la tabla pedido.

Para el contraste crítico se agregó además la afirmación deliberadamente imprecisa «el costo final 143.13 representa 143.13 ms», marcada como incorrecta en la tabla de la Parte 3.